

RESUMIND

AI Resume Analyzer

An intelligent platform to analyze, optimize, and elevate your resume
for modern hiring systems and ATS compatibility.

Technical Project Report · April 2026

Capstone Project · Full-Stack AI Application

Node.js	Next.js	TypeScript	Prisma	PostgreSQL	Clerk	Groq AI
---------	---------	------------	--------	------------	-------	---------

Team Capstone 2026	Stack Full-Stack · AI · TypeScript	Backend Node.js · Express · Prisma	Frontend Next.js · Tailwind CSS
--------------------	---------------------------------------	---------------------------------------	------------------------------------

Table of Contents

- 1. Project Overview
- 2. Key Features
- 3. System Architecture
- 4. Module Breakdown
 - 4.1 ATS Score Checker
 - 4.2 Resume Enhancer
 - 4.3 Resume Builder
 - 4.4 Authentication & Dashboard
- 5. OOP & SOLID Principles
- 6. Design Patterns
- 7. Database Schema
- 8. API Reference
- 9. Tech Stack
- 10. Application Flow
- 11. Installation & Setup

1.

Project Overview

Resumind is an intelligent, full-stack web application that helps job seekers analyze, optimize, and enhance their resumes using AI. Instead of manually editing resumes or guessing what recruiters want, Resumind provides a **structured, data-driven approach** to resume building — evaluating content, improving formatting, enhancing keyword relevance, and generating actionable insights to increase ATS (Applicant Tracking System) success rates.

The platform simplifies resume optimization into a seamless workflow: upload a resume, receive AI-driven enhancements tailored to a specific role, and export a polished PDF — all within a single authenticated interface.

Module	Description
ATS Score Checker	Upload a PDF resume and receive an instant compatibility score, keyword gap analysis, and section-by-section improvement feedback.
Resume Enhancer	Upload resume + paste a job description + select a role. Groq AI rewrites the resume to match the role, then exports a downloadable PDF.
Resume Builder	Create a resume from scratch via a manual structured form or an AI chat interface. Supports live preview and PDF export.
Dashboard	Authenticated user area displaying all saved resumes with download, delete, and re-enhance options.

2.

Key Features

Core Capabilities

- **AI Resume Enhancement** — Groq LLM (llama3-8b-8192) rewrites resume content to match a specific job description with role-specific optimizations.
- **ATS Score Analysis** — Evaluates resume against ATS criteria, produces a 0–100 compatibility score, identifies keyword gaps, and provides section feedback.
- **Resume Builder** — Guided manual form or AI chat interface for building resumes from scratch with live preview.
- **Role-Based Optimization** — Software Engineer, Product Manager, and Data Scientist enhancers each apply tailored AI instructions.
- **Secure Authentication** — Email-only login via Clerk with session management. No passwords stored in our database.
- **Persistent Dashboard** — All enhanced and built resumes are saved per user with full management controls.
- **PDF Export** — All resumes are exported as well-formatted, downloadable PDF files.

Planned Capabilities

- Cover Letter Generator — personalized cover letters from resume + job description
- Interview Preparation — AI-based mock interview questions tailored to the role
- Job Matching — suggest relevant job listings based on resume content
- Advanced Export — Word (.docx) and ATS-ready plain-text formats

Platform Comparison

Domain	Traditional Approach	Resumind
Resume Writing	Manual editing, no guidance	AI-assisted optimization per role
ATS Compatibility	Guesswork	Data-driven keyword and score analysis
Feedback	None or recruiter-dependent	Real-time AI insights per section
Efficiency	Hours of manual work	Automated end-to-end pipeline
Export	Word files, manual formatting	Instant PDF generation

3.

System Architecture

Resumind is organized as a monorepo containing a **Next.js frontend** and a **Node.js/Express backend**. The backend follows a strict layered architecture with clean separation between HTTP handling, business logic, AI integration, and data persistence. All inter-layer dependencies are governed by TypeScript interfaces, enabling loose coupling and easy component substitution.

Architecture Layers

Layer	Key Components	Responsibility
HTTP Layer	Express Routes, Controllers	Parse requests, validate inputs, send responses. Zero business logic.
Business Logic	ResumeService, UserService	Orchestrate the enhancement/analysis pipeline. No HTTP or DB code.
AI Layer	GroqAIProvider, BaseRoleEnhancer subclasses	Call Groq LLM with role-specific prompts and return enhanced text.
Parsing Layer	PDFResumeParser, ParserFactory	Extract plain text from uploaded PDF buffers.
Generation Layer	PDFGenerator	Convert enhanced text into formatted downloadable PDF files.
Data Layer	UserRepository, ResumeRepository, Prisma	Persist all users, resumes, and results to Neon DB (PostgreSQL).
Auth Layer	Clerk, clerkAuth middleware	Verify Clerk JWT tokens. Email-only authentication enforced via Clerk.

Resume Enhancement — Request Lifecycle

01	Request received User submits PDF + job description + role via POST /api/resume/enhance.
02	Auth verification clerkAuth middleware verifies the Clerk session JWT. Attaches userId to request.
03	File validation uploadMiddleware (Multer) confirms file is PDF and under 10 MB. Stores as memory buffer.
04	Controller delegates ResumeController reads req fields and delegates to ResumeService. No logic in controller.
05	PDF parsing PDFResumeParser.parse() extracts plain text from the PDF buffer using pdf-parse.

06	Role enhancer selected RoleEnhancerFactory.create(role, ai) returns the correct enhancer subclass.
07	AI enhancement Enhancer calls GroqAIProvider.enhance() with role-specific system prompt + job description.
08	LLM response Groq returns the enhanced resume text (llama3-8b-8192, max 2048 tokens).
09	PDF generation PDFGenerator writes the enhanced text to /uploads/resume-{uuid}.pdf.
10	Database persistence ResumeRepository saves originalText, enhancedText, jobDescription, pdfUrl to Neon DB.
11	Response returned Controller sends { id, pdfUrl, enhancedText } to frontend for preview and download.

4.

Module Breakdown

4.1 ATS Score Checker

The ATS Score Checker allows users to upload their PDF resume and receive an instant compatibility analysis. The backend extracts resume text, runs it through the ATSEngine, and returns a structured JSON report including an overall score, keyword gaps, and section-by-section improvement suggestions.

Component	Responsibility
ParserFactory	Detects file type and returns the correct parser implementation (PDF or DOCX).
PDFResumeParser	Extracts plain text from PDF buffers using the pdf-parse library.
TextProcessor	Cleans and normalizes extracted resume text before analysis.
ATSEngine	Scores resume against ATS criteria and identifies keyword gaps and section weaknesses.
ATSController	HTTP handler — reads the uploaded file, delegates to ATSEngine, returns JSON response.
ATSRouter	Defines /api/ats routes and registers upload and error middleware.

Endpoint:

```
POST /api/ats/analyze → form-data: resume (PDF file)
```

Returns: { score: number (0–100), gaps: string[], feedback: { section: string, suggestion: string }[] }

4.2 Resume Enhancer

The Resume Enhancer is the core AI module. Users upload a PDF resume, paste a job description, and select a target role. The system uses the role-specific enhancer subclass to build a focused AI prompt, calls Groq's LLM, and returns an enhanced resume as both text and a downloadable PDF saved to the user's dashboard.

Role Slug	Enhancer Class	AI Emphasis
software-engineer	SoftwareEngineerEnhancer	Languages, frameworks, system design, scalability, engineering impact
product-manager	ProductManagerEnhancer	Product launches, DAU/MAU metrics, A/B testing, revenue and growth impact
data-scientist	DataScientistEnhancer	ML models, Python/R, data pipelines, statistical analysis, business impact

Method	Endpoint	Auth	Description
GET	/api/resume/roles	No	Returns list of supported roles for the frontend dropdown.
POST	/api/resume/enhance	Yes	Upload PDF + enhance. Returns { id, pdfUrl, enhancedText }.
GET	/api/resume/my-resumes	Yes	Returns all saved resumes for the authenticated user.
GET	/api/resume/:id	Yes	Returns a single resume by its UUID.

4.3 Resume Builder (Create)

The Resume Builder offers two creation modes. The **Manual path** lets users select a template and fill a structured form, with a live preview panel updating in real time. The **AI Chat path** lets users describe their experience conversationally and have the AI generate structured content. Both paths produce a PDF export and recommend running the ATS Score Checker on the result.

- **Manual Path:** Choose template → fill resume form → live preview → download PDF or save to dashboard.
- **AI Chat Path:** Chat with AI → AI builds resume content → live preview → download PDF.
- Both paths enforce ATS-friendly formatting automatically.
- After generation, the system recommends the ATS Score Checker as a next step.

4.4 Authentication & Dashboard

Authentication is handled entirely by Clerk, configured for email-only login. On first login, the frontend calls **POST /api/user/sync** to create the user record in Neon DB, using the Clerk user ID as the primary key — no password is ever stored. The clerkAuth middleware verifies every protected request using the Clerk Backend SDK.

Endpoint	Auth	Description
POST /api/user/sync	No	Syncs Clerk user into Neon DB on first login. Uses upsert — safe to call on every login.
GET /api/user/me	Yes	Returns the current authenticated user's profile from the database.

The dashboard lists all saved resumes with direct **Download**, **Delete**, and **Re-enhance** options. Navigation provides access to all four modules.

5.

OOP & SOLID Principles

The backend is architected with explicit adherence to all four OOP pillars and three SOLID principles. Every design decision maps directly to a specific file and class in the codebase, making the architecture straightforward to explain and verify.

Four Pillars of OOP

Pillar	Where in Codebase	How It Is Applied
Encapsulation	GroqAIProvider PDFResumeParser PDFGenerator Repositories	Each class hides its internal library completely. GroqAIProvider hides the Groq SDK. PDFResumeParser hides pdf-parse. Repositories hide Prisma. The rest of the app calls a single method and receives a result.
Abstraction	IAIProvider IResumeParser IPDFGenerator IResumeRepository IUserRepository	Five TypeScript interfaces define contracts (WHAT must happen) without specifying implementation (HOW it happens). Services depend only on these interfaces.
Inheritance	SoftwareEngineerEnhancer ProductManagerEnhancer DataScientistEnhancer	All three extend BaseRoleEnhancer and inherit the enhance() template method. Each subclass overrides only getRoleName() and getRoleContext(). Common logic is written exactly once in the parent.
Polymorphism	RoleEnhancerFactory ResumeService	roleEnhancer.enhance() is called identically regardless of which subclass was returned by the factory. The correct role-specific behavior executes automatically at runtime — the caller never checks the type.

SOLID Principles

Principle	Applied In	Evidence
S — Single Responsibility	Every file in src/	PDFResumeParser only parses. PDFGenerator only creates PDFs. ResumeController only handles HTTP requests and responses. ResumeService only orchestrates the pipeline. Each class has exactly one reason to change.
O — Open/Closed	BaseRoleEnhancer RoleEnhancerFactory	Adding a new role requires creating ONE new file (e.g. DesignerEnhancer.ts extending BaseRoleEnhancer) and adding ONE line to the factory map. BaseRoleEnhancer and ResumeService are never modified.
D — Dependency Inversion	ResumeService constructor	ResumeService receives IResumeParser, IAIProvider, IPDFGenerator, and IResumeRepository — all interfaces. To swap Groq for OpenAI, change one line in ResumeController. ResumeService never changes. High-level modules depend on abstractions, not on concrete classes.

6.

Design Patterns

Pattern	Location	Purpose & Explanation
Singleton	src/config/prisma.ts src/config/groq.ts	Ensures exactly one PrismaClient and one Groq client exist across the entire app. Without this, each request would create a new DB connection, quickly exhausting the connection pool and crashing the server.
Factory	RoleEnhancerFactory	Creates the correct role enhancer subclass based on a role string (e.g. 'software-engineer' → SoftwareEngineerEnhancer). The caller never instantiates enhancers directly, keeping the creation logic centralized in one place.
Repository	UserRepository ResumeRepository	Isolates all database queries in dedicated repository classes. Services never write Prisma queries directly. If the ORM changes from Prisma to something else, only the repository files need updating.
Template Method	BaseRoleEnhancer .enhance()	Defines the skeleton of the enhancement algorithm in the base class. Step 1 (get role context) is customized by each subclass. Step 2 (call AI provider) is always identical. This avoids code duplication across all role enhancers.
Composition Root	ResumeController constructor	The single location in the app where all concrete classes are instantiated and injected into the service. Everywhere else in the codebase uses interfaces only. This is where Dependency Inversion is made concrete.

7.

Database Schema

The database is hosted on **Neon DB** (serverless PostgreSQL) and managed via **Prisma ORM v7**. The Prisma client is generated to a custom output path (`src/generated/prisma`) using a `prisma.config.ts` configuration file. The schema consists of two core models with a one-to-many relationship.

User Model

Field	Type	Notes
id	String (PK)	Clerk user ID (e.g. <code>user_2abc123</code>). No auto-generation — Clerk owns identity.
email	String (UNIQUE)	User email address. Email-only login enforced via Clerk dashboard settings.
createdAt	DateTime	Account creation timestamp. Auto-set to <code>NOW()</code> on insert.
resumes	Resume[] (Relation)	One-to-many relation. One user can own many resume records.

Resume Model

Field	Type	Notes
id	String UUID (PK)	Auto-generated UUID. Default: <code>uuid()</code> .
userId	String (FK)	References <code>User.id</code> . CASCADE delete — user deleted → resumes deleted.
originalText	String	Plain text extracted from the uploaded PDF by <code>PDFResumeParser</code> .
enhancedText	String	AI-generated enhanced resume text returned by Groq.
jobDescription	String	Job description pasted by the user in the enhance form.
pdfUrl	String	Relative path to the generated PDF file (e.g. <code>/uploads/resume-uuid.pdf</code>).
createdAt	DateTime	Enhancement timestamp. Auto-set to <code>NOW()</code> on insert.

Prisma Schema

```
generator client {
  provider = "prisma-client-js"
  output   = "../src/generated/prisma"
}

datasource db {
  provider = "postgresql"
```

```
url      = env("DATABASE_URL")    // pooled connection
directUrl = env("DIRECT_URL")      // direct connection for migrations
}

model User {
  id      String    @id
  email   String    @unique
  createdAt DateTime @default(now())
  resumes Resume[]
  @@map("users")
}

model Resume {
  id          String    @id @default(uuid())
  userId      String
  originalText String
  enhancedText String
  jobDescription String
  pdfUrl      String
  createdAt   DateTime @default(now())
  user User @relation(fields: [userId], references: [id], onDelete: Cascade)
  @@map("resumes")
}
```

8.

API Reference

All protected endpoints require a valid Clerk session token in the Authorization header: **Authorization: Bearer <clerk_session_token>**. The clerkAuth middleware verifies this token before the request reaches the controller. Token is obtained from useAuth() on the Next.js frontend.

Method	Endpoint	Auth	Request Body / Params	Response
POST	/api/user/sync	No	JSON: { id, email }	{ success: true, message }
GET	/api/user/me	Yes	—	{ success: true, data: User }
GET	/api/resume/roles	No	—	{ success: true, data: string[] }
POST	/api/resume/enhance	Yes	form-data: resume (PDF file), jobDescription, role	{ success: true, data: { id, pdfUrl, enhancedText } }
GET	/api/resume/my-resumes	Yes	—	{ success: true, data: Resume[] }
GET	/api/resume/:id	Yes	URL param: id (UUID)	{ success: true, data: Resume }
POST	/api/ats/analyze	No	form-data: resume (PDF file)	{ score, gaps: string[], feedback: object[] }

Error Responses

All endpoints return consistent error shapes. 400 for bad input, 401 for missing/invalid token, 404 for not found, 500 for server errors.

```
{ "error": "Descriptive error message here." }
```

9.

Tech Stack

Domain	Technology	Version	Purpose
Frontend	Next.js + React	14+	App routing, server components, and React UI pages.
Styling	Tailwind CSS	3+	Utility-first CSS framework for all styling.
Animation	Framer Motion	Latest	Page transitions and micro-interaction animations.
Backend	Node.js + Express	v22 / v5	REST API server handling all backend routes.
Language	TypeScript	6.0	Static typing across both frontend and backend.
AI Provider	Groq SDK	1.1+	LLM inference using llama3-8b-8192 model.
Authentication	Clerk	3.4+	Email-only auth, JWT session tokens, user management.
ORM	Prisma	7.8	Type-safe database queries and schema management.
Database	Neon DB (PostgreSQL)	Latest	Serverless PostgreSQL with connection pooling.
File Upload	Multer	2.1+	Multipart form data handling with memory storage.
PDF Parsing	pdf-parse	2.4+	Extract text content from uploaded PDF files.
PDF Generation	PDFKit	0.18+	Generate formatted downloadable PDF resumes.
Dev Server	ts-node-dev	2.0+	TypeScript hot-reload development server.

10.

Application Flow

The following describes the complete end-to-end user journey through Resumind, from the landing page through to export and dashboard management.

01	Landing Page User arrives at the Resumind landing page. Core features and value proposition are presented.
02	Authentication User signs up or logs in via Clerk (email only). On first login, frontend calls POST /api/user/sync to create the user record in Neon DB.
03	Dashboard Authenticated user is redirected to the dashboard. Previously saved resumes are listed. Navigation provides access to Create, ATS Checker, Enhancer, and Profile.
04	Choose Module User selects one of three modules: Resume Builder, ATS Score Checker, or Resume Enhancer.
05	ATS Score Checker User uploads PDF → backend extracts text → ATSEngine scores and analyses → results show score gauge, keyword gaps, and section feedback → CTA links to the Enhancer.
06	Resume Enhancer User uploads PDF + pastes job description + selects role → AI pipeline runs (parse → enhance → generate PDF) → enhanced resume displayed with keyword additions, ATS score, and download link → option to save to dashboard.
07	Resume Builder User selects manual template or AI chat mode → fills or generates resume content → live preview updates → ATS score shown inline → download PDF or save to dashboard.
08	Dashboard — Saved Resumes All saved resumes appear in the dashboard with Download, Delete, and Re-enhance options.
09	Export User downloads the optimized resume as a PDF at any point in the flow.

11.

Installation & Setup

Prerequisites

- Node.js v18 or higher (v22 recommended)
- npm (bundled with Node.js)
- Neon DB account — free tier at neon.tech
- Groq API key — free at console.groq.com
- Clerk account — free at clerk.com, configured for email-only login

Setup Steps

Step 1: Clone the repository

```
git clone https://github.com/yourusername/resumind.git
cd resumind
```

Step 2: Install backend dependencies

```
cd backend && npm install
```

Step 3: Install frontend dependencies

```
cd ../frontend && npm install
```

Step 4: Configure environment

```
Create backend/.env:

PORT=5001
DATABASE_URL=prisma_pooled_url_from_neon
DIRECT_URL=direct_url_from_neon
GROQ_API_KEY=gsk_XXXXXXXXXXXX
CLERK_SECRET_KEY=sk_test_XXXXXXXXXXXX
```

Step 5: Generate Prisma client

```
cd backend && npx prisma generate
```

Step 6: Push schema to Neon DB

```
npx prisma db push
```

Step 7: Start backend

```
npm run dev # starts on port 5001
```

Step 8: Start frontend

```
cd ../frontend && npm run dev # starts on port 3000
```

Port Reference

Service	Local Port
Frontend (Next.js)	3000
Backend (Express API)	5001
Prisma Studio (optional)	5555